

CSE 515: Multimedia and Web Databases

Image Features, Vector Models and Similarity Measures

Phase 1 Project Report

Author

Gokul Vasudeva

gvasude2@asu.edu

September 9, 2023

Group 15

Caleb Panikulam, Gokul Vasudeva, Joshua Martin Noronha, Kiran Sthanasubramonian,
Sandipan De, Shankar Harinarayanan

Abstract

In an increasingly digital visual landscape, the need for effective content recommendation/retrieval systems is paramount. The primary objective of this project is to create a powerful tool capable of efficiently searching and retrieving relevant images from the Caltech-101 dataset. One way to accomplish this is by generating feature descriptors using various vector models in the form of image vectors, and comparing a query image's vector representation to that of a pre-calculated vector from the dataset using certain similarity/distance measures. We utilize five vector models; a color model, a gradient/texture model, and the outputs of three layers in the ResNet-50 pre-trained convolutional neural network.

Keywords - Color moments, Histogram of oriented gradients (HOG), ResNet-50, Intersection similarity, Cosine Similarity, Correlation distance, Image retrieval, Caltech-101.

Introduction

In an era characterized by an abundance of visual data, this project aims to explore the potential of utilizing Histogram of Oriented Gradients (HOG), color moments, and the ResNet-50 deep convolutional neural network as feature extractors, with the end goal of powering an image retrieval system. Image retrieval is inherently a subjective task^[1], where the semantics of what a user is requesting is difficult to understand and capture. The three methodologies chosen for this project are each unique, and considering all three allows us to amortize the inadequacies of one model with another; HOG, known for its ability to capture shape and texture information, color moments, which are effective at encoding color characteristics. Pre-trained ResNet-50, is a widely used deep learning architecture, and can extract high-level features and representations from images^[2], making it valuable for various tasks such as image classification and object detection, which are characteristically related to image retrieval. This project envisions applications across different domains, including image search engines, object recognition systems, and content recommendation platforms.

Terminology

1. Color moments - Measures that characterize color distribution in an image in the same way that central moments uniquely describe a probability distribution^[3].
2. HoG - Histogram of oriented gradients is a feature descriptor used in computer vision and image processing for the purpose of object detection. The technique counts occurrences of gradient orientation in localized portions of an image^[4].
3. ResNet-50 - A Residual Neural Network (a.k.a. Residual Network, ResNet) is a deep learning model in which the weight layers learn residual functions with reference to the layer inputs. ResNet-50 is a network that contains 50 layers^[2].
4. Pre-trained - A network that has had its weights adjusted on a prior learning task, in this case, ImageNet1k-V2.
5. Caltech-101 - Pictures of objects belonging to 101 categories, with about 40 to 800 images per category^[5].

Goal description

The goal of this project is to generate feature vectors for images in the Caltech-101 dataset through various vector models, store them in some database, and to allow users to query for similar images in the dataset by providing an image.

This project is broken down into 3 tasks:

1. Implement a program which, given an image ID and one of the following feature models, visualizes the image and then extracts and prints (in a human readable form) the corresponding feature descriptors:

1. Color moments (CM10x10)
2. Histograms of oriented gradients (HOG)
3. ResNet-AvgPool-1024
4. ResNet-Layer3-1024
5. ResNet-FC-1000

2. Implement a program which extracts and stores feature descriptors for all the images in the data set.

3. Implement a program which, given an image ID and a value “k”, returns and visualizes the most similar k images based on each of the visual model -you will select the appropriate distance/similarity measure for each feature model. For each match, also list the corresponding distance/similarity score.

Assumptions

The project was developed with the intention of being used with the Caltech-101 dataset, as such, the design decisions only considered this case. Since this dataset only has 8677 images, it is feasible to keep all feature vectors in memory with the 5 vector models under consideration. The system requires RGB images to function. Consider color moments, the feature vector of a non-RGB image would vary in content or length if an image's color channels do not match. As such, all images are converted to RGB if they are not already. The formulas used for the implementations for color moments and HOG are from the lecture notes.

Description of the proposed implementation

The solution heavily utilizes PyTorch's TorchVision, NumPy, and SciPy frameworks. At the beginning of each task, the Caltech-101 dataset is downloaded/re-verified using TorchVision's wrapper around the dataset. TorchVision combines images into a single list with a range of [0, 8676]. It does so by lexicographically considering each category and each image within that category.

Each image is resized to WxH of 300x100, before being converted to a grid of patches of 10x10, with each patch being 30x10.

For color moments, the mean, standard deviation and skew of each channel in each patch is calculated, resulting in 3 features that encode the color's histogram, and this is done per channel (RGB), per patch. This results in a feature vector of length $3 \times 3 \times 10 \times 10 = 900$.

For HOG, images are first converted to grayscale after being resized to 300x100, and gradients are calculated by applying $\langle -1, 0, 1 \rangle$ horizontally and $\langle -1, 0, 1 \rangle^T$ vertically, as G_x and G_y respectively. Magnitudes for these are calculated as the square root of sum of squares, and the directions are calculated using quadrant aware arctan. This results in a signed direction in radians, which are converted to a degree range of $[0, 360)$. These are then binned into 9 bins, with each bin corresponding to a multiple of 40 degrees.

A strategy for magnitude weighted binning is followed such that when considering the adjacent bins for any direction, such that the magnitude is distributed to the adjacent bin in proportion to how close the direction is to the bin. Eg:

pixel(x, y) has magnitude: 72, direction: 50. Since the adjacent bins are at 40 and 80 degrees, $72 * |80 - 50|/40 = 54$ is added to bin 40, and $72 * |40 - 50|/40 = 18$ is added to bin 80.

This binning strategy is applied to each patch, resulting in $9 \times 10 \times 10 = 900$ dimension feature vector.

For the features extracted from the ResNet-50 pre-trained network, the input image is resized to 224x224, and hooks are added at the outputs of avgpool, layer3, and fc1000, and transformed as follows:

AvgPool outputs 1x2048, we have to average every 2 values sequentially to obtain a 1024 dimension vector. Layer3 outputs 1x1024x14x14, we take the mean of each 14x14 slice,

resulting in a 1024 dimension vector. Fc1000 outputs 1x1000, which is taken as a 1000 dimension vector.

For Task 1, the above is computed and displayed, along with relevant images and histograms to showcase the system. Task 2 involves running the vector generation sequence across every image in the dataset, and storing them as a pickle object mapping the image ID to a feature vector for each vector model. Task 3 involves generating the feature vectors for a new query image ID, and loading the previously generated pickles into memory. A distance measure is used for each vector model to compute how “far” the query image is from any other image in the dataset, and the images with the K smallest distances are showcased.

Choosing a similarity measure

Choosing an appropriate similarity measure for each vector model is difficult, as it is inherently subjective. It cannot easily be ascertained what a user means by “similar”, as each feature model captures different features. For example, the color model would capture color semantics well, but not things like texture or abstract objects. In some cases, the user might care the most about the color semantics, while in other cases, color could be irrelevant to what they consider as similar.

The implementation of the project uses several distances, which is related to similarity as $1 - d_norm^{[6]}$. The strategy used for selecting an appropriate distance measure involved 3 parts:

1. Iterating over each image, and each model using Wasserstein distance (EMD), Intersection distance, Correlation distance, Cosine distance, Euclidean distance, Manhattan distance, and Jacquard distance, and counting how many of the top 10 images of each model belonged to the query image’s category (which is exposed by the Caltech-101 dataset). This was used to narrow down candidate distance measures for each model.
2. Using the subset of distance measures per model to setup trials where the author subjectively annotated the performance of each measure for 100 randomly generated image IDs.
3. Mapping the semantic applications and high level objective of each distance measure with the semantic output of each vector model.

Considering all three of the above strategies, the following distance measure were chosen for each vector model:

A. Color moments - Manhattan distance (City-block distance): This distance can be imagined as the length needed to move between two points in a grid where you can only move up, down, left or right^[7]. This measure is sensitive to differences along each dimension of the feature vectors, capturing the total absolute difference between corresponding moments, which makes it great for assessing differences in color distribution in terms of absolute deviations. Since it does not square the differences like Euclidean distance, it is less susceptible outliers when many dimensions differ.

B. HoG - Correlation similarity: This measures the linear relationship between HOG feature vectors, and performs well with features that have a strong linear correlation or dependency in their gradient orientation distributions, taking into account directionality^[8].

C. ResNet50 AvgPool, ResNet50 Layer3, ResNet50 FC1000 - Cosine similarity: Cosine was suitable, since the output of the ResNet layers are normalized, reducing the importance of magnitude. It is good at discerning semantic or structural similarity rather than their absolute feature values^[9]. It also works well with sparse feature spaces such as those found in CNNs.

Interface specifications

Task 1 has been presented in the form of a Jupyter Notebook with an input prompt for accepting the image ID. The outputs for this task are visualized within the notebook, including histograms for colors, as well as histograms for binned gradients as well as visualizations of the magnitudes of gradients. Task 2 is a Python script, which iterates over all images in the dataset as described in the previous section, and outputs the feature vectors for each model as a binary encoded .pt file stored in Code/tensor_database/. Task 3 has been presented in the form of a Jupyter Notebook with input prompts for accepting the image ID as well as the value of K. The outputs for this task are visualized within the notebook.

Installation and execution instructions

The system was developed using Python 3.11.5.

1. It is recommended to setup a virtual environment within the project folder:

```
python3 -m venv .venv
```

```
source venv/bin/activate
```

2. Install requirements within venv:

```
pip install -r requirements.txt
```

3. Either have an IDE which supports opening Jupyter Notebooks respecting choosing the virtual environment as the interpreter (such as VSCode), or start jupyter notebook from within the venv:

```
jupyter notebook
```

4. This should open a new browser tab with the codebase. Navigate to Code/<task_no>.ipynb to open a notebook, and follow the instructions.

Results

The results of task 3 for each of the query images with IDs: 0, 880, 2500, 5122, 8676 with K=10 are present within the Outputs/ directory that accompanies this report. They have also been added on to the end of this document.

Related work

N. Dalal and B. Triggs. "Histograms of Oriented Gradients for Human Detection." In International Conference on Computer Vision & Pattern Recognition, 2005.

J. Zhang and M. Marszalek. "SIFT Meets Color: A Case Study for Object Recognition in the Lab." In European Conference on Computer Vision, 2006.

Y. Wang et al. "Learning Fine-Grained Image Similarity with Deep Ranking." In Conference on Computer Vision and Pattern Recognition, 2014.

X. Zhang et al. "Combining Color and Texture Information for Image Retrieval." In IEEE Transactions on Image Processing, 2005.

Conclusions

Image query based image retrieval is an inherently ambiguous task, and while established methods and state of the art may perform better, it is nonetheless worthwhile to explore well-researched topics to better understand them. We experimented with various types of feature extractors, transformation of those features into reasonably sized feature vectors using various techniques, and leveraged similarity measures chosen through empirical, subjective and intuitive analysis for ascertaining the closeness to a query image. It was found that subjectively, the query performance was better using most similarity measures and the feature vectors generated through ResNet-50, and as such, future steps in this direction might want to further explore CNNs as feature extractors for the purpose of image retrieval.

Bibliography

1. *Content-based image retrieval*. (2023, July 24). Wikipedia. https://en.wikipedia.org/wiki/Content-based_image_retrieval
2. He, K. (2015, December 10). *Deep Residual Learning for Image Recognition*. arXiv.org. <https://arxiv.org/abs/1512.03385>
3. *Color moments*. (2023, July 24). Wikipedia. https://en.wikipedia.org/wiki/Color_moments
4. *Histogram of oriented gradients*. (2023, January 28). Wikipedia. https://en.wikipedia.org/wiki/Histogram_of_oriented_gradients
5. Li, F.-F., Andreoto, M., Ranzato, M., & Perona, P. (2022). *Caltech 101 (1.0) [Data set]*. CaltechDATA. <https://doi.org/10.22002/D1.20086>
6. *Similarity measure*. (2023, August 13). Wikipedia. https://en.wikipedia.org/wiki/Similarity_measure
7. *Taxicab geometry*. (2023, July 6). Wikipedia. https://en.wikipedia.org/wiki/Taxicab_geometry
8. *Correlation*. (2023, September 7). Wikipedia. <https://en.wikipedia.org/wiki/Correlation>
9. *Cosine similarity*. (2023, August 9). Wikipedia. https://en.wikipedia.org/wiki/Cosine_similarity

Appendix

Roles of group members - This phase of the project is an individual project with individual deliverables. As such, all code was written solely by the author, and the contents of this report were written solely by the author, with relevant sources cited in this report as well as in the code via comments.

Discussions were had with other teammates on their approaches to the interface, their implementations of vector models, and their chosen similarity measures, but this was merely a discussion to understand their thought process, and to help those members who were stuck, or had results that deviated greatly from what the majority of what other team members had.

I personally took no help from other teammates with respect to implementation or understanding, but I did assist others.

Instructions:

1. Run task_2.py to generate the tensor database if not already generated.
2. Press Run All (or restart kernel and run all cells).
3. You will be prompted to provide an image ID and a k value.

Task 3: Implement a program which, given an image ID and a value “k”, returns and visualizes the most similar k images based on each of the visual model –you will select the appropriate distance/similarity measure for each feature model.
For each match, also list the corresponding distance/similarity score.

```
In [ ]: IMG_ID = int(input("Provide an Image ID in the Caltech101 dataset in range [0, 8676]."))

K = int(input("Enter K, the number of similar images K to find for each feature model."))
```

```
In [ ]: # Initialize dataset, downloads into <git_root>/Code/caltech101 if not present. Untracked by git.

import os
from torchvision.datasets import Caltech101

dataset = Caltech101(root=os.path.abspath(""), download=True)
```

Files already downloaded and verified

```
In [ ]: # Find the k most similar images for a given Image ID for each feature model.

image, category = dataset[IMG_ID][0], dataset.categories[dataset[IMG_ID][1]]

if image.mode != 'RGB':
    print("WARNING: The provided image does not have RGB channels, the image will be converted to RGB.")
    image = image.convert('RGB')

print("Input image:\nImage ID: ", IMG_ID)
display(image)
print("Category:", category)
```

Input image:
Image ID: 0



Category: Faces

```
In [ ]: # Resize to 300x100 and partition to grid.  
  
image300x100 = image.resize((300, 100))  
  
from utils.utilities import partition_to_grid  
  
grid = partition_to_grid(image300x100, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

```
In [ ]: # Color moments (CM10x10).  
  
# Find K most similar images by color moments.  
  
from feature_extractors.color_moments import get_color_vector  
  
color_vector = get_color_vector(grid)  
  
# Load color tensor database into memory.  
  
from utils.utilities import tensor_loader
```

```

color_tensor_dict = tensor_loader('color_tensors.pt', os.path.abspath(""))

from utils.utilities import top_k_ranker

# Manhattan distance (City-block distance): This distance can be imagined as the length needed
# to move between two points in a grid where you can only move up, down, left or right. This
# measure is sensitive to differences along each dimension of the feature vectors, capturing
# the total absolute difference between corresponding moments, which makes it great for
# assessing differences in color distribution in terms of absolute deviations. Since it
# does not square the differences like Euclidean distance, it is less susceptible outliers
# when many dimensions differ.

# We use Earth Mover's Distance (Wasserstein distance) as color moments can be thought
# of as probability distributions over color space. EMD is capable of comparing distributions
# with different shapes and scales, color moments consider the spatial arrangement of colors,
# not just the frequency of colors. EMD takes this spatial information into account by
# considering how color probability mass is transported from one distribution to another.

from scipy.spatial.distance import cityblock

print("Similar images by Color Moments using distance: Manhattan (cityblock)")

top_k_by_color = top_k_ranker(K, color_vector, color_tensor_dict, cityblock)

for i in range(len(top_k_by_color)):

    id = top_k_by_color[i][1]
    distance = top_k_by_color[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])

```

Similar images by Color Moments using distance: Manhattan (cityblock)

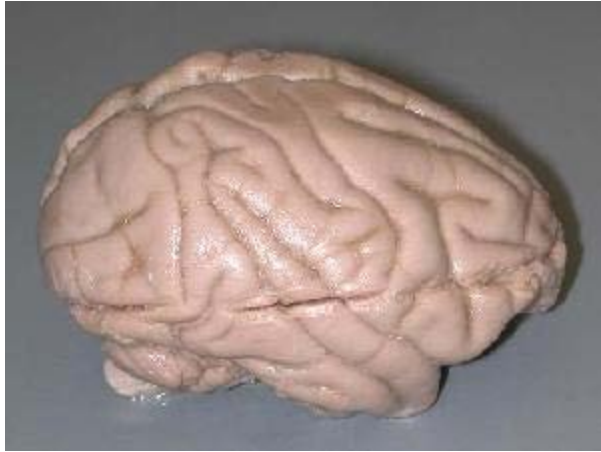
1. Image ID: 0 Distance: 0.0



2. Image ID: 435 Distance: 20543.707



3. Image ID: 3196 Distance: 20989.852



4. Image ID: 5160

Distance: 21095.695



5. Image ID: 5058

Distance: 21099.266



6. Image ID: 4 Distance: 21197.104



7. Image ID: 7615 Distance: 21968.61



8. Image ID: 275 Distance: 22091.668



9. Image ID: 4255 Distance: 22128.664



10. Image ID: 7258

Distance: 22196.605



```
In [ ]: # Convert to grayscale and partition to grid.  
  
from torchvision.transforms import Grayscale  
  
gs_image = Grayscale()(image300x100)
```

```
gs_grid = partition_to_grid(gs_image, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

```
In [ ]: # Histograms of oriented gradients (HOG).

# Find K most similar images by HOG.

from feature_extractors.HOG import get_hog_vector

hog_vector = get_hog_vector(gs_grid)

# Load hog tensor database into memory.

hog_tensor_dict = tensor_loader('hog_tensors.pt', os.path.abspath(""))

# Correlation similarity: This measures the linear relationship between HOG feature vectors,
# and performs well with features that have a strong linear correlation or dependency in
# their gradient orientation distributions, taking into account directionality.

from scipy.spatial.distance import correlation

print("Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation")

top_k_by_hog = top_k_ranker(K, hog_vector, hog_tensor_dict, correlation)

for i in range(len(top_k_by_hog)):

    id = top_k_by_hog[i][1]
    distance = top_k_by_hog[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation
1.      Image ID: 0          Distance: 0.0
```



2. Image ID: 7260

Distance: 0.1772551461243549



3. Image ID: 6384

Distance: 0.18155921487948135



4. Image ID: 7684

Distance: 0.1828890761653944



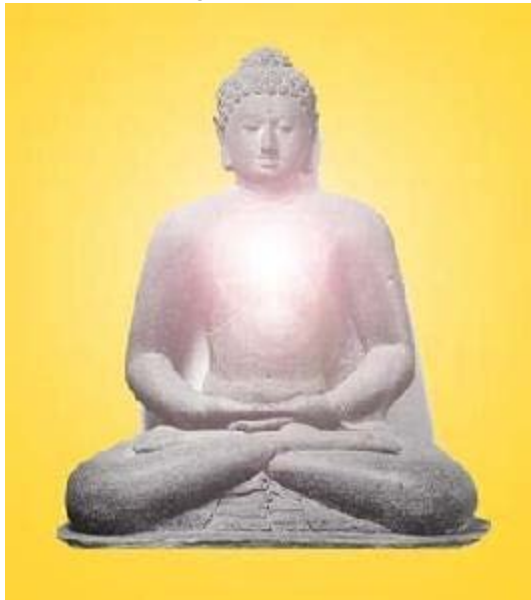
5. Image ID: 7621

Distance: 0.1832137524684716



6. Image ID: 3271

Distance: 0.18473419767448518



7. Image ID: 5655

Distance: 0.18711687502127605



8. Image ID: 5886

Distance: 0.18741789194374237



9. Image ID: 7000

Distance: 0.18751472764420007



10. Image ID: 5900

Distance: 0.18756641471349578



```
In [ ]: # Resize original image to 224x224 and convert to float tensor.
```

```
from torchvision.transforms import transforms
```

```
image224x224 = image.resize((224, 224))
```

```
img_tensor = transforms.Compose([transforms.ToTensor()])(image224x224)
```

```
In [ ]: # ResNet pre-processing.
```

```
# Find K most similar images by each of the 3 layers chosen from ResNet-50.
```

```

from feature_extractors.resnet_50 import get_resnet50_feature_vectors

avgpool_vector, layer3_vector, fc1000_vector = get_resnet50_feature_vectors(img_tensor)

# Load each tensor database into memory

avgpool_tensor_dict = tensor_loader('avgpool_tensors.pt', os.path.abspath(""))
layer3_tensor_dict = tensor_loader('layer3_tensors.pt', os.path.abspath(""))
fc1000_tensor_dict = tensor_loader('fc1000_tensors.pt', os.path.abspath(""))

```

```

In [ ]: # ResNet-AvgPool-1024.

# Find K most similar images by ResNet-AvgPool-1024.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

from scipy.spatial.distance import cosine

print("Similar images by ResNet-AvgPool-1024 using distance: Cosine")

top_k_by_avgpool = top_k_ranker(K, avgpool_vector, avgpool_tensor_dict, cosine)

for i in range(len(top_k_by_avgpool)):

    id = top_k_by_avgpool[i][1]
    distance = top_k_by_avgpool[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])

```

```

Similar images by ResNet-AvgPool-1024 using distance: Cosine
1.      Image ID: 0           Distance: 0

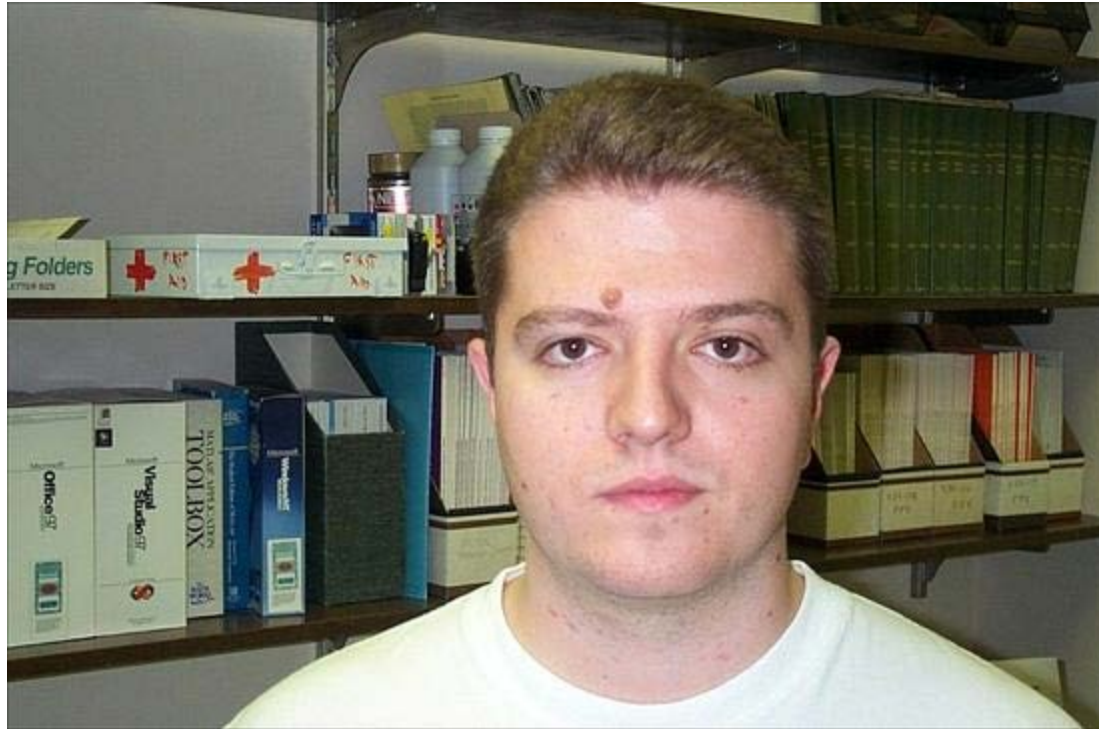
```




2. Image ID: 435 Distance: 0.14790743589401245



3. Image ID: 51 Distance: 0.18542933464050293



4. Image ID: 156 Distance: 0.19760167598724365



5. Image ID: 220 Distance: 0.20157980918884277



6. Image ID: 152 Distance: 0.2020755410194397



7. Image ID: 34 Distance: 0.2084168791770935



8. Image ID: 18 Distance: 0.213627099999084473



9. Image ID: 27 Distance: 0.2262974977493286



10. Image ID: 832 Distance: 0.22704273462295532



```
In [ ]: # ResNet-Layer3-1024.  
  
# Find K most similar images by ResNet-Layer3-1024.  
  
# Cosine similarity: We use Cosine since the output of the ResNet layer is  
# normalized, reducing the importance of magnitude. It is good at discerning  
# semantic or structural similarity rather than their absolute feature values.  
# It also works well with sparse feature spaces such as those found in CNNs.  
  
print("Similar images by ResNet-Layer3-1024 using distance: Cosine")  
  
top_k_by_layer3 = top_k_ranker(K, layer3_vector, layer3_tensor_dict, cosine)  
  
for i in range(len(top_k_by_layer3)):  
    id = top_k_by_layer3[i][1]  
    distance = top_k_by_layer3[i][0]  
  
    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)  
    display(dataset[id][0])
```

Similar images by ResNet-Layer3-1024 using distance: Cosine

1. Image ID: 0 Distance: 0



2. Image ID: 435 Distance: 0.02364814281463623



3. Image ID: 27 Distance: 0.028365373611450195



4. Image ID: 156 Distance: 0.02884727716445923



5. Image ID: 4 Distance: 0.03036510944366455



6. Image ID: 59 Distance: 0.0312839150428772



7. Image ID: 220 Distance: 0.03134727478027344



8. Image ID: 18 Distance: 0.03184032440185547



9. Image ID: 135 Distance: 0.03284412622451782



10. Image ID: 231 Distance: 0.0330316424369812



```
In [ ]: # ResNet-FC-1000.

# Find K most similar images by ResNet-FC-1000.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

print("Similar images by ResNet-FC-1000 using distance: Cosine")

top_k_by_fc1000 = top_k_ranker(K, fc1000_vector, fc1000_tensor_dict, cosine)

for i in range(len(top_k_by_fc1000)):

    id = top_k_by_fc1000[i][1]
    distance = top_k_by_fc1000[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by ResNet-FC-1000 using distance: Cosine

1. Image ID: 0 Distance: 0



2. Image ID: 435 Distance: 0.19422084093093872



3. Image ID: 18 Distance: 0.20374107360839844



4. Image ID: 156 Distance: 0.20885878801345825



5. Image ID: 220 Distance: 0.22211605310440063



6. Image ID: 49 Distance: 0.2309613823890686



7. Image ID: 152 Distance: 0.2348458170890808



8. Image ID: 51 Distance: 0.2357337474822998



9. Image ID: 469 Distance: 0.24090707302093506



10. Image ID: 55 Distance: 0.24337798357009888



Instructions:

1. Run task_2.py to generate the tensor database if not already generated.
2. Press Run All (or restart kernel and run all cells).
3. You will be prompted to provide an image ID and a k value.

Task 3: Implement a program which, given an image ID and a value "k", returns and visualizes the most similar k images based on each of the visual model –you will select the appropriate distance/similarity measure for each feature model. For each match, also list the corresponding distance/similarity score.

```
In [ ]: IMG_ID = int(input("Provide an Image ID in the Caltech101 dataset in range [0, 8676]."))  
  
K = int(input("Enter K, the number of similar images K to find for each feature model."))
```

```
In [ ]: # Initialize dataset, downloads into <git_root>/Code/caltech101 if not present. Untracked by git.  
  
import os  
from torchvision.datasets import Caltech101  
  
dataset = Caltech101(root=os.path.abspath(""), download=True)
```

Files already downloaded and verified

```
In [ ]: # Find the k most similar images for a given Image ID for each feature model.  
  
image, category = dataset[IMG_ID][0], dataset.categories[dataset[IMG_ID][1]]  
  
if image.mode != 'RGB':  
    print("WARNING: The provided image does not have RGB channels, the image will be converted to RGB.")  
    image = image.convert('RGB')  
  
print("Input image:\nImage ID: ", IMG_ID)  
display(image)  
print("Category:", category)
```

Input image:
Image ID: 880



Category: Leopards

```
In [ ]: # Resize to 300x100 and partition to grid.

image300x100 = image.resize((300, 100))

from utils.utilities import partition_to_grid

grid = partition_to_grid(image300x100, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

```
In [ ]: # Color moments (CM10x10).

# Find K most similar images by color moments.

from feature_extractors.color_moments import get_color_vector

color_vector = get_color_vector(grid)

# Load color tensor database into memory.

from utils.utilities import tensor_loader

color_tensor_dict = tensor_loader('color_tensors.pt', os.path.abspath(""))

from utils.utilities import top_k_ranker

# Manhattan distance (City-block distance): This distance can be imagined as the length needed
# to move between two points in a grid where you can only move up, down, left or right. This
# measure is sensitive to differences along each dimension of the feature vectors, capturing
# the total absolute difference between corresponding moments, which makes it great for
# assessing differences in color distribution in terms of absolute deviations. Since it
# does not square the differences like Euclidean distance, it is less susceptible outliers
# when many dimensions differ.
```

```
# We use Earth Mover's Distance (Wasserstein distance) as color moments can be thought
# of as probability distributions over color space. EMD is capable of comparing distributions
# with different shapes and scales, color moments consider the spatial arrangement of colors,
# not just the frequency of colors. EMD takes this spatial information into account by
# considering how color probability mass is transported from one distribution to another.
```

```
from scipy.spatial.distance import cityblock

print("Similar images by Color Moments using distance: Manhattan (cityblock)")

top_k_by_color = top_k_ranker(K, color_vector, color_tensor_dict, cityblock)

for i in range(len(top_k_by_color)):

    id = top_k_by_color[i][1]
    distance = top_k_by_color[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by Color Moments using distance: Manhattan (cityblock)

1. Image ID: 880 Distance: 0.0



2. Image ID: 980 Distance: 12883.973



3. Image ID: 970 Distance: 12894.973



4. Image ID: 974

Distance: 12919.006



5. Image ID: 987

Distance: 13368.09



6. Image ID: 925

Distance: 13505.451



7. Image ID: 887

Distance: 13672.375



8. Image ID: 874

Distance: 13687.33



9. Image ID: 876

Distance: 13732.93



10. Image ID: 976

Distance: 13764.921



```
In [ ]: # Convert to grayscale and partition to grid.

from torchvision.transforms import Grayscale

gs_image = Grayscale()(image300x100)

gs_grid = partition_to_grid(gs_image, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

```
In [ ]: # Histograms of oriented gradients (HOG).

# Find K most similar images by HOG.

from feature_extractors.HOG import get_hog_vector

hog_vector = get_hog_vector(gs_grid)

# Load hog tensor database into memory.

hog_tensor_dict = tensor_loader('hog_tensors.pt', os.path.abspath(""))

# Correlation similarity: This measures the linear relationship between HOG feature vectors,
# and performs well with features that have a strong linear correlation or dependency in
# their gradient orientation distributions, taking into account directionality.

from scipy.spatial.distance import correlation

print("Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation")

top_k_by_hog = top_k_ranker(K, hog_vector, hog_tensor_dict, correlation)

for i in range(len(top_k_by_hog)):

    id = top_k_by_hog[i][1]
    distance = top_k_by_hog[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation
1.      Image ID: 880          Distance: 0.0
```




2. Image ID: 1044

Distance: 0.1108569742828649



3. Image ID: 980

Distance: 0.11876509688290737



4. Image ID: 7406

Distance: 0.12034339240493097



5. Image ID: 944

Distance: 0.12036418127448689



6. Image ID: 1000

Distance: 0.12048191374948791



7. Image ID: 987

Distance: 0.12250058086032545



8. Image ID: 974

Distance: 0.12269712751484674



9. Image ID: 887

Distance: 0.12503799602553078



10. Image ID: 911

Distance: 0.12519870864032556



```
In [ ]: # Resize original image to 224x224 and convert to float tensor.
```

```
from torchvision.transforms import transforms
```

```
image224x224 = image.resize((224, 224))
```

```
img_tensor = transforms.Compose([transforms.ToTensor()])(image224x224)
```

```
In [ ]: # ResNet pre-processing.
```

```
# Find K most similar images by each of the 3 layers chosen from ResNet-50.
```

```
from feature_extractors.resnet_50 import get_resnet50_feature_vectors
```

```
avgpool_vector, layer3_vector, fc1000_vector = get_resnet50_feature_vectors(img_tensor)
```

```
# Load each tensor database into memory
```

```
avgpool_tensor_dict = tensor_loader('avgpool_tensors.pt', os.path.abspath(''))
```

```
layer3_tensor_dict = tensor_loader('layer3_tensors.pt', os.path.abspath(""))
fc1000_tensor_dict = tensor_loader('fc1000_tensors.pt', os.path.abspath(""))
```

```
In [ ]: # ResNet-AvgPool-1024.

# Find K most similar images by ResNet-AvgPool-1024.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

from scipy.spatial.distance import cosine

print("Similar images by ResNet-AvgPool-1024 using distance: Cosine")

top_k_by_avgpool = top_k_ranker(K, avgpool_vector, avgpool_tensor_dict, cosine)

for i in range(len(top_k_by_avgpool)):

    id = top_k_by_avgpool[i][1]
    distance = top_k_by_avgpool[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by ResNet-AvgPool-1024 using distance: Cosine

1. Image ID: 880 Distance: 0



2. Image ID: 980 Distance: 0.02858734130859375



3. Image ID: 976

Distance: 0.0936480164527893



4. Image ID: 1022

Distance: 0.10950839519500732



5. Image ID: 898

Distance: 0.11175507307052612



6. Image ID: 996

Distance: 0.1154336929321289



7. Image ID: 922

Distance: 0.11590331792831421



8. Image ID: 998

Distance: 0.11629730463027954



9. Image ID: 896

Distance: 0.11795306205749512



10. Image ID: 886

Distance: 0.11834180355072021



```
In [ ]: # ResNet-Layer3-1024.

# Find K most similar images by ResNet-Layer3-1024.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

print("Similar images by ResNet-Layer3-1024 using distance: Cosine")

top_k_by_layer3 = top_k_ranker(K, layer3_vector, layer3_tensor_dict, cosine)

for i in range(len(top_k_by_layer3)):

    id = top_k_by_layer3[i][1]
    distance = top_k_by_layer3[i][0]

    print(str(i + 1) + ". ", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by ResNet-Layer3-1024 using distance: Cosine

1. Image ID: 880 Distance: 0



2. Image ID: 980 Distance: 0.002772510051727295



3. Image ID: 876

Distance: 0.01864105463027954



4. Image ID: 976

Distance: 0.021041393280029297



5. Image ID: 1010

Distance: 0.02151542901992798



6. Image ID: 910

Distance: 0.022718727588653564



7. Image ID: 875

Distance: 0.023593544960021973



8. Image ID: 993

Distance: 0.023832857608795166



9. Image ID: 893

Distance: 0.024298250675201416



10. Image ID: 1022

Distance: 0.025303542613983154



```
In [ ]: # ResNet-FC-1000.

# Find K most similar images by ResNet-FC-1000.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

print("Similar images by ResNet-FC-1000 using distance: Cosine")

top_k_by_fc1000 = top_k_ranker(K, fc1000_vector, fc1000_tensor_dict, cosine)

for i in range(len(top_k_by_fc1000)):

    id = top_k_by_fc1000[i][1]
    distance = top_k_by_fc1000[i][0]

    print(str(i + 1) + ". ", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by ResNet-FC-1000 using distance: Cosine

1. Image ID: 880 Distance: 0



2. Image ID: 980 Distance: 0.02508002519607544



3. Image ID: 976

Distance: 0.1025659441947937



4. Image ID: 999

Distance: 0.10723358392715454



5. Image ID: 922

Distance: 0.10978174209594727



6. Image ID: 1022

Distance: 0.11090725660324097



7. Image ID: 995

Distance: 0.11403411626815796



8. Image ID: 898

Distance: 0.11512356996536255



9. Image ID: 876

Distance: 0.11669296026229858



10. Image ID: 996

Distance: 0.11676293611526489



Instructions:

1. Run task_2.py to generate the tensor database if not already generated.
2. Press Run All (or restart kernel and run all cells).
3. You will be prompted to provide an image ID and a k value.

Task 3: Implement a program which, given an image ID and a value "k", returns and visualizes the most similar k images based on each of the visual model –you will select the appropriate distance/similarity measure for each feature model. For each match, also list the corresponding distance/similarity score.

```
In [ ]: IMG_ID = int(input("Provide an Image ID in the Caltech101 dataset in range [0, 8676]."))  
  
K = int(input("Enter K, the number of similar images K to find for each feature model."))
```

```
In [ ]: # Initialize dataset, downloads into <git_root>/Code/caltech101 if not present. Untracked by git.  
  
import os  
from torchvision.datasets import Caltech101  
  
dataset = Caltech101(root=os.path.abspath(""), download=True)
```

Files already downloaded and verified

```
In [ ]: # Find the k most similar images for a given Image ID for each feature model.  
  
image, category = dataset[IMG_ID][0], dataset.categories[dataset[IMG_ID][1]]  
  
if image.mode != 'RGB':  
    print("WARNING: The provided image does not have RGB channels, the image will be converted to RGB.")  
    image = image.convert('RGB')  
  
print("Input image:\nImage ID: ", IMG_ID)  
display(image)  
print("Category:", category)
```

Input image:
Image ID: 2500



Category: airplanes

```
In [ ]: # Resize to 300x100 and partition to grid.

image300x100 = image.resize((300, 100))

from utils.utilities import partition_to_grid

grid = partition_to_grid(image300x100, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

```
In [ ]: # Color moments (CM10x10).

# Find K most similar images by color moments.

from feature_extractors.color_moments import get_color_vector

color_vector = get_color_vector(grid)

# Load color tensor database into memory.

from utils.utilities import tensor_loader

color_tensor_dict = tensor_loader('color_tensors.pt', os.path.abspath(""))

from utils.utilities import top_k_ranker

# Manhattan distance (City-block distance): This distance can be imagined as the length needed
# to move between two points in a grid where you can only move up, down, left or right. This
# measure is sensitive to differences along each dimension of the feature vectors, capturing
# the total absolute difference between corresponding moments, which makes it great for
```

```

# assessing differences in color distribution in terms of absolute deviations. Since it
# does not square the differences like Euclidean distance, it is less susceptible outliers
# when many dimensions differ.

# We use Earth Mover's Distance (Wasserstein distance) as color moments can be thought
# of as probability distributions over color space. EMD is capable of comparing distributions
# with different shapes and scales, color moments consider the spatial arrangement of colors,
# not just the frequency of colors. EMD takes this spatial information into account by
# considering how color probability mass is transported from one distribution to another.

from scipy.spatial.distance import cityblock

print("Similar images by Color Moments using distance: Manhattan (cityblock)")

top_k_by_color = top_k_ranker(K, color_vector, color_tensor_dict, cityblock)

for i in range(len(top_k_by_color)):

    id = top_k_by_color[i][1]
    distance = top_k_by_color[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])

```

Similar images by Color Moments using distance: Manhattan (cityblock)

1. Image ID: 2500 Distance: 0.0



2. Image ID: 2301 Distance: 10972.34



3. Image ID: 2519 Distance: 11094.308



4. Image ID: 2524 Distance: 11336.744



5. Image ID: 2308 Distance: 12073.262



6. Image ID: 2196 Distance: 12535.328



7. Image ID: 2245 Distance: 12603.198



8. Image ID: 2410 Distance: 13100.132



9. Image ID: 5007

Distance: 13128.953



10. Image ID: 2670

Distance: 13422.051



```
In [ ]: # Convert to grayscale and partition to grid.  
  
from torchvision.transforms import Grayscale  
  
gs_image = Grayscale()(image300x100)  
  
gs_grid = partition_to_grid(gs_image, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

```
In [ ]: # Histograms of oriented gradients (HOG).  
  
# Find K most similar images by HOG.  
  
from feature_extractors.HOG import get_hog_vector  
  
hog_vector = get_hog_vector(gs_grid)  
  
# Load hog tensor database into memory.  
  
hog_tensor_dict = tensor_loader('hog_tensors.pt', os.path.abspath(""))  
  
# Correlation similarity: This measures the linear relationship between HOG feature vectors,  
# and performs well with features that have a strong linear correlation or dependency in  
# their gradient orientation distributions, taking into account directionality.  
  
from scipy.spatial.distance import correlation  
  
print("Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation")
```

```
top_k_by_hog = top_k_ranker(K, hog_vector, hog_tensor_dict, correlation)

for i in range(len(top_k_by_hog)):
    id = top_k_by_hog[i][1]
    distance = top_k_by_hog[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation

1. Image ID: 2500 Distance: 0.0



2. Image ID: 2121 Distance: 0.0661546313127187



3. Image ID: 2308 Distance: 0.06905893472781599



4. Image ID: 2094 Distance: 0.07435084787559465



5. Image ID: 2673 Distance: 0.07991670203757606



6. Image ID: 5038 Distance: 0.08053593607760079



7. Image ID: 2167 Distance: 0.08069762336058162



8. Image ID: 2609 Distance: 0.08195859934637628



9. Image ID: 2301 Distance: 0.08204215381630864



10. Image ID: 2191 Distance: 0.08445441642668183



```
In [ ]: # Resize original image to 224x224 and convert to float tensor.
```

```
from torchvision.transforms import transforms
```

```
image224x224 = image.resize((224, 224))
```

```
img_tensor = transforms.Compose([transforms.ToTensor()])(image224x224)
```

```
In [ ]: # ResNet pre-processing.
```

```
# Find K most similar images by each of the 3 layers chosen from ResNet-50.
```

```
from feature_extractors.resnet_50 import get_resnet50_feature_vectors
```

```
avgpool_vector, layer3_vector, fc1000_vector = get_resnet50_feature_vectors(img_tensor)
```

```
# Load each tensor database into memory
```



```
avgpool_tensor_dict = tensor_loader('avgpool_tensors.pt', os.path.abspath(""))
layer3_tensor_dict = tensor_loader('layer3_tensors.pt', os.path.abspath(""))
fc1000_tensor_dict = tensor_loader('fc1000_tensors.pt', os.path.abspath(""))
```

```
In [ ]: # ResNet-AvgPool-1024.

# Find K most similar images by ResNet-AvgPool-1024.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

from scipy.spatial.distance import cosine

print("Similar images by ResNet-AvgPool-1024 using distance: Cosine")

top_k_by_avgpool = top_k_ranker(K, avgpool_vector, avgpool_tensor_dict, cosine)

for i in range(len(top_k_by_avgpool)):

    id = top_k_by_avgpool[i][1]
    distance = top_k_by_avgpool[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by ResNet-AvgPool-1024 using distance: Cosine

1. Image ID: 2500 Distance: 0



2. Image ID: 2308 Distance: 0.06326800584793091



3. Image ID: 2167 Distance: 0.08668988943099976



4. Image ID: 2527 Distance: 0.09355634450912476



5. Image ID: 2254 Distance: 0.10535889863967896



6. Image ID: 2301 Distance: 0.11197084188461304



7. Image ID: 2398 Distance: 0.11582589149475098



8. Image ID: 2226 Distance: 0.11904072761535645



9. Image ID: 2412 Distance: 0.1205405592918396



10. Image ID: 2670 Distance: 0.12307322025299072



```
In [ ]: # ResNet-Layer3-1024.  
  
# Find K most similar images by ResNet-Layer3-1024.  
  
# Cosine similarity: We use Cosine since the output of the ResNet layer is  
# normalized, reducing the importance of magnitude. It is good at discerning  
# semantic or structural similarity rather than their absolute feature values.  
# It also works well with sparse feature spaces such as those found in CNNs.  
  
print("Similar images by ResNet-Layer3-1024 using distance: Cosine")  
  
top_k_by_layer3 = top_k_ranker(K, layer3_vector, layer3_tensor_dict, cosine)  
  
for i in range(len(top_k_by_layer3)):  
    id = top_k_by_layer3[i][1]  
    distance = top_k_by_layer3[i][0]  
  
    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)  
    display(dataset[id][0])
```

```
Similar images by ResNet-Layer3-1024 using distance: Cosine  
1.      Image ID: 2500          Distance: 0
```



2. Image ID: 2519 Distance: 0.012333095073699951



3. Image ID: 2413 Distance: 0.013264119625091553



4. Image ID: 2217 Distance: 0.015680968761444092



5. Image ID: 2624 Distance: 0.016152143478393555



6. Image ID: 2569 Distance: 0.016269266605377197



7. Image ID: 2527 Distance: 0.016563475131988525



8. Image ID: 2625 Distance: 0.01666736602783203



9. Image ID: 2509 Distance: 0.01810145378112793



10. Image ID: 2411 Distance: 0.01848888397216797



```
In [ ]: # ResNet-FC-1000.  
        # Find K most similar images by ResNet-FC-1000.
```

```
# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

print("Similar images by ResNet-FC-1000 using distance: Cosine")

top_k_by_fc1000 = top_k_ranker(K, fc1000_vector, fc1000_tensor_dict, cosine)

for i in range(len(top_k_by_fc1000)):

    id = top_k_by_fc1000[i][1]
    distance = top_k_by_fc1000[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by ResNet-FC-1000 using distance: Cosine

1. Image ID: 2500 Distance: 0



2. Image ID: 2308 Distance: 0.05851387977600098



3. Image ID: 2167 Distance: 0.06570607423782349



4. Image ID: 2301 Distance: 0.07090252637863159



5. Image ID: 2203 Distance: 0.07465589046478271



6. Image ID: 2254 Distance: 0.07529681921005249



7. Image ID: 2253 Distance: 0.07565945386886597



8. Image ID: 2670 Distance: 0.07674413919448853



9. Image ID: 2527 Distance: 0.07712769508361816



10. Image ID: 2252 Distance: 0.07915341854095459



Instructions:

1. Run task_2.py to generate the tensor database if not already generated.
2. Press Run All (or restart kernel and run all cells).
3. You will be prompted to provide an image ID and a k value.

Task 3: Implement a program which, given an image ID and a value "k", returns and visualizes the most similar k images based on each of the visual model –you will select the appropriate distance/similarity measure for each feature model. For each match, also list the corresponding distance/similarity score.

```
In [ ]: IMG_ID = int(input("Provide an Image ID in the Caltech101 dataset in range [0, 8676]."))

K = int(input("Enter K, the number of similar images K to find for each feature model."))
```

```
In [ ]: # Initialize dataset, downloads into <git_root>/Code/caltech101 if not present. Untracked by git.

import os
from torchvision.datasets import Caltech101

dataset = Caltech101(root=os.path.abspath(""), download=True)
```

Files already downloaded and verified

```
In [ ]: # Find the k most similar images for a given Image ID for each feature model.

image, category = dataset[IMG_ID][0], dataset.categories[dataset[IMG_ID][1]]

if image.mode != 'RGB':
    print("WARNING: The provided image does not have RGB channels, the image will be converted to RGB.")
    image = image.convert('RGB')

print("Input image:\nImage ID: ", IMG_ID)
display(image)
print("Category:", category)
```

WARNING: The provided image does not have RGB channels, the image will be converted to RGB.
 Input image:
 Image ID: 5122



Category: garfield

```
In [ ]: # Resize to 300x100 and partition to grid.  
  
image300x100 = image.resize((300, 100))  
  
from utils.utilities import partition_to_grid  
  
grid = partition_to_grid(image300x100, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

```
In [ ]: # Color moments (CM10x10).  
  
# Find K most similar images by color moments.  
  
from feature_extractors.color_moments import get_color_vector  
  
color_vector = get_color_vector(grid)  
  
# Load color tensor database into memory.  
  
from utils.utilities import tensor_loader  
  
color_tensor_dict = tensor_loader('color_tensors.pt', os.path.abspath(''))
```

```
from utils.utilities import top_k_ranker

# Manhattan distance (City-block distance): This distance can be imagined as the length needed
# to move between two points in a grid where you can only move up, down, left or right. This
# measure is sensitive to differences along each dimension of the feature vectors, capturing
# the total absolute difference between corresponding moments, which makes it great for
# assessing differences in color distribution in terms of absolute deviations. Since it
# does not square the differences like Euclidean distance, it is less susceptible outliers
# when many dimensions differ.

# We use Earth Mover's Distance (Wasserstein distance) as color moments can be thought
# of as probability distributions over color space. EMD is capable of comparing distributions
# with different shapes and scales, color moments consider the spatial arrangement of colors,
# not just the frequency of colors. EMD takes this spatial information into account by
# considering how color probability mass is transported from one distribution to another.

from scipy.spatial.distance import cityblock

print("Similar images by Color Moments using distance: Manhattan (cityblock)")

top_k_by_color = top_k_ranker(K, color_vector, color_tensor_dict, cityblock)

for i in range(len(top_k_by_color)):

    id = top_k_by_color[i][1]
    distance = top_k_by_color[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by Color Moments using distance: Manhattan (cityblock)

1. Image ID: 5122 Distance: 0.0



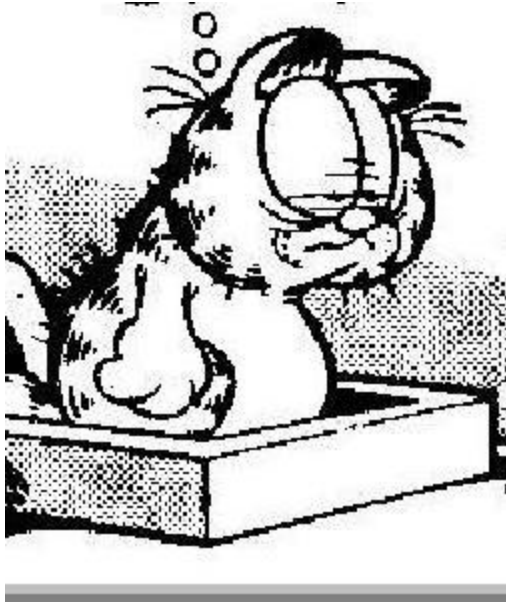
2. Image ID: 6454

Distance: 35980.383



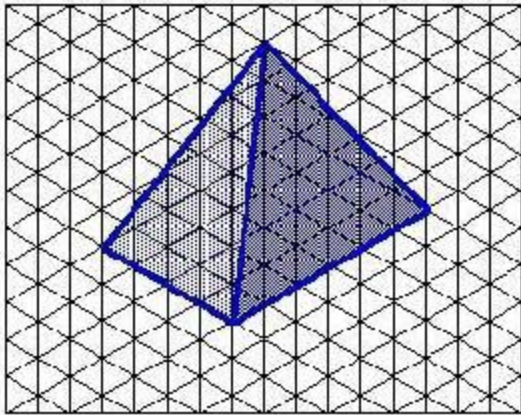
3. Image ID: 5132

Distance: 36270.492



4. Image ID: 6989

Distance: 38271.375



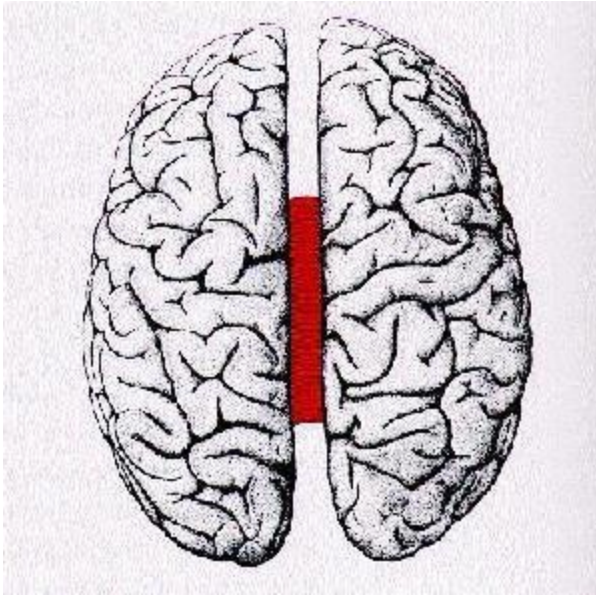
5. Image ID: 8209

Distance: 38983.95



6. Image ID: 3155

Distance: 39237.914



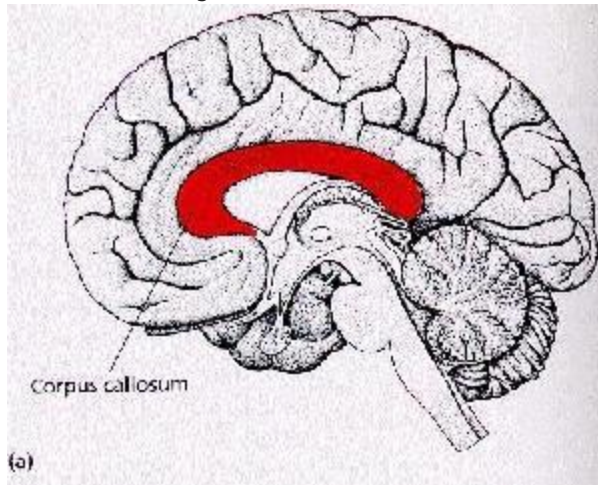
7. Image ID: 7497

Distance: 39320.727



8. Image ID: 3153

Distance: 39748.87



(a)

9. Image ID: 8536

Distance: 39790.188



10. Image ID: 5137

Distance: 39922.633



```
In [ ]: # Convert to grayscale and partition to grid.
```

```
from torchvision.transforms import Grayscale
```

```
gs_image = Grayscale()(image300x100)

gs_grid = partition_to_grid(gs_image, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

```
In [ ]: # Histograms of oriented gradients (HOG).

# Find K most similar images by HOG.

from feature_extractors.HOG import get_hog_vector

hog_vector = get_hog_vector(gs_grid)

# Load hog tensor database into memory.

hog_tensor_dict = tensor_loader('hog_tensors.pt', os.path.abspath(""))

# Correlation similarity: This measures the linear relationship between HOG feature vectors,
# and performs well with features that have a strong linear correlation or dependency in
# their gradient orientation distributions, taking into account directionality.

from scipy.spatial.distance import correlation

print("Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation")

top_k_by_hog = top_k_ranker(K, hog_vector, hog_tensor_dict, correlation)

for i in range(len(top_k_by_hog)):

    id = top_k_by_hog[i][1]
    distance = top_k_by_hog[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation
1.      Image ID: 5122                Distance: 0.0
```



2. Image ID: 3777

Distance: 0.1684781403242568



3. Image ID: 7686

Distance: 0.1724295083705072



Photo: Ed Jackson

4. Image ID: 7281

Distance: 0.17562480133592773



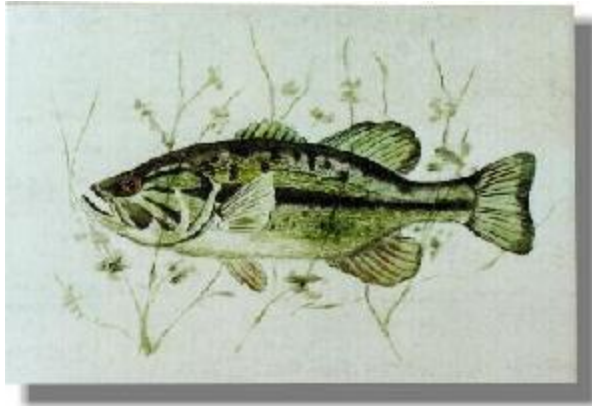
5. Image ID: 5899

Distance: 0.17570036623356555



6. Image ID: 2897

Distance: 0.1812149767199902



7. Image ID: 3300

Distance: 0.18197529353446296



8. Image ID: 6057

Distance: 0.18380253699033222



9. Image ID: 6035

Distance: 0.18473298327785848



10. Image ID: 8150

Distance: 0.18640012403866713



```
In [ ]: # Resize original image to 224x224 and convert to float tensor.
```

```
from torchvision.transforms import transforms
```

```
image224x224 = image.resize((224, 224))
```

```
img_tensor = transforms.Compose([transforms.ToTensor()]) (image224x224)
```

```
In [ ]: # ResNet pre-processing.

# Find K most similar images by each of the 3 layers chosen from ResNet-50.

from feature_extractors.resnet_50 import get_resnet50_feature_vectors

avgpool_vector, layer3_vector, fc1000_vector = get_resnet50_feature_vectors(img_tensor)

# Load each tensor database into memory

avgpool_tensor_dict = tensor_loader('avgpool_tensors.pt', os.path.abspath(""))
layer3_tensor_dict = tensor_loader('layer3_tensors.pt', os.path.abspath(""))
fc1000_tensor_dict = tensor_loader('fc1000_tensors.pt', os.path.abspath(""))
```

```
In [ ]: # ResNet-AvgPool-1024.

# Find K most similar images by ResNet-AvgPool-1024.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

from scipy.spatial.distance import cosine

print("Similar images by ResNet-AvgPool-1024 using distance: Cosine")

top_k_by_avgpool = top_k_ranker(K, avgpool_vector, avgpool_tensor_dict, cosine)

for i in range(len(top_k_by_avgpool)):

    id = top_k_by_avgpool[i][1]
    distance = top_k_by_avgpool[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by ResNet-AvgPool-1024 using distance: Cosine

1. Image ID: 5122 Distance: 0



2. Image ID: 5137

Distance: 0.18765169382095337



3. Image ID: 5112

Distance: 0.21680545806884766



4. Image ID: 7474

Distance: 0.22793978452682495



5. Image ID: 7471

Distance: 0.2319655418395996



6. Image ID: 2888

Distance: 0.2446938157081604



7. Image ID: 7479

Distance: 0.2543797492980957



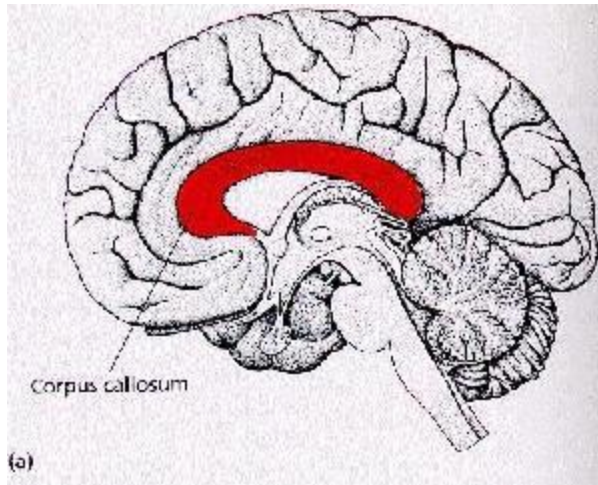
8. Image ID: 5132

Distance: 0.25515806674957275



9. Image ID: 3153

Distance: 0.25874292850494385



10. Image ID: 7497

Distance: 0.270865261554718



```
In [ ]: # ResNet-Layer3-1024.
```

```
# Find K most similar images by ResNet-Layer3-1024.
```

```
# Cosine similarity: We use Cosine since the output of the ResNet layer is  
# normalized, reducing the importance of magnitude. It is good at discerning  
# semantic or structural similarity rather than their absolute feature values.  
# It also works well with sparse feature spaces such as those found in CNNs.
```

```
print("Similar images by ResNet-Layer3-1024 using distance: Cosine")

top_k_by_layer3 = top_k_ranker(K, layer3_vector, layer3_tensor_dict, cosine)

for i in range(len(top_k_by_layer3)):

    id = top_k_by_layer3[i][1]
    distance = top_k_by_layer3[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by ResNet-Layer3-1024 using distance: Cosine

1. Image ID: 5122 Distance: 0



2. Image ID: 5132

Distance: 0.03839331865310669



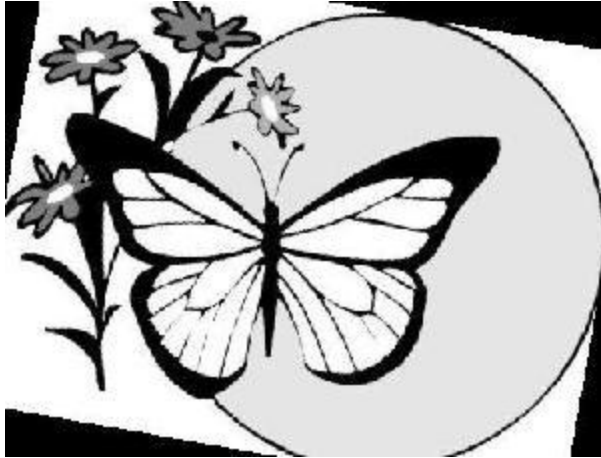
3. Image ID: 5137

Distance: 0.05274897813796997



4. Image ID: 3383

Distance: 0.05594545602798462



5. Image ID: 2888

Distance: 0.05623418092727661



6. Image ID: 7497

Distance: 0.05633091926574707



7. Image ID: 5112

Distance: 0.059560179710388184



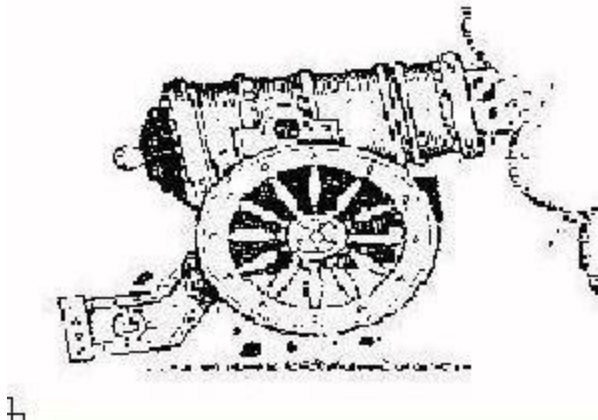
8. Image ID: 7471

Distance: 0.06032222509384155



9. Image ID: 3518

Distance: 0.060375869274139404



10. Image ID: 7475

Distance: 0.06101977825164795



```
In [ ]: # ResNet-FC-1000.

# Find K most similar images by ResNet-FC-1000.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

print("Similar images by ResNet-FC-1000 using distance: Cosine")

top_k_by_fc1000 = top_k_ranker(K, fc1000_vector, fc1000_tensor_dict, cosine)

for i in range(len(top_k_by_fc1000)):

    id = top_k_by_fc1000[i][1]
    distance = top_k_by_fc1000[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by ResNet-FC-1000 using distance: Cosine
1.      Image ID: 5122          Distance: 0
```



2. Image ID: 5137

Distance: 0.16487550735473633



3. Image ID: 7474

Distance: 0.19566476345062256



4. Image ID: 7471

Distance: 0.20404183864593506



5. Image ID: 5112

Distance: 0.20469021797180176



6. Image ID: 2888

Distance: 0.2152014970779419



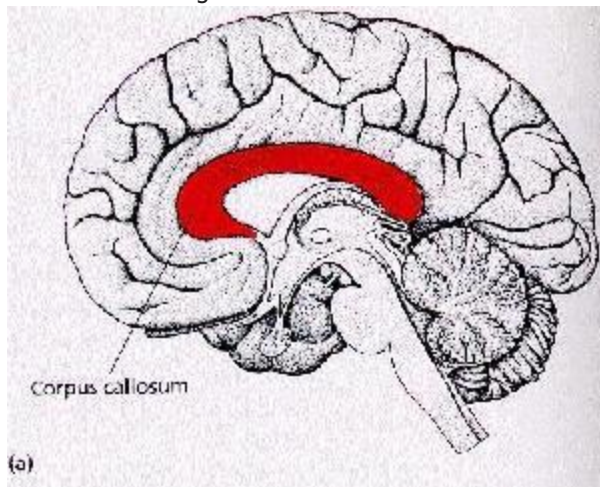
7. Image ID: 7475

Distance: 0.23861569166183472



8. Image ID: 3153

Distance: 0.24297446012496948



9. Image ID: 7497

Distance: 0.24322742223739624



10. Image ID: 5136

Distance: 0.25195860862731934



Instructions:

1. Run task_2.py to generate the tensor database if not already generated.
2. Press Run All (or restart kernel and run all cells).
3. You will be prompted to provide an image ID and a k value.

Task 3: Implement a program which, given an image ID and a value "k", returns and visualizes the most similar k images based on each of the visual model –you will select the appropriate distance/similarity measure for each feature model. For each match, also list the corresponding distance/similarity score.

```
In [ ]: IMG_ID = int(input("Provide an Image ID in the Caltech101 dataset in range [0, 8676]."))

K = int(input("Enter K, the number of similar images K to find for each feature model.))
```

```
In [ ]: # Initialize dataset, downloads into <git_root>/Code/caltech101 if not present. Untracked by git.

import os
from torchvision.datasets import Caltech101

dataset = Caltech101(root=os.path.abspath(""), download=True)
```

Files already downloaded and verified

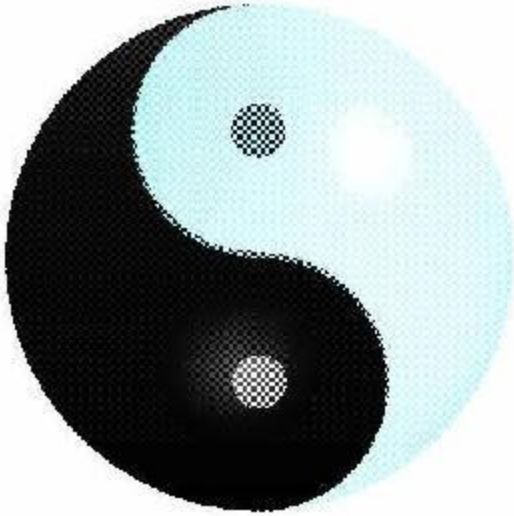
```
In [ ]: # Find the k most similar images for a given Image ID for each feature model.

image, category = dataset[IMG_ID][0], dataset.categories[dataset[IMG_ID][1]]

if image.mode != 'RGB':
    print("WARNING: The provided image does not have RGB channels, the image will be converted to RGB.")
    image = image.convert('RGB')

print("Input image:\nImage ID: ", IMG_ID)
display(image)
print("Category:", category)
```

Input image:
Image ID: 8676



Category: yin_yang

```
In [ ]: # Resize to 300x100 and partition to grid.  
  
image300x100 = image.resize((300, 100))  
  
from utils.utilities import partition_to_grid  
  
grid = partition_to_grid(image300x100, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

```
In [ ]: # Color moments (CM10x10).  
  
# Find K most similar images by color moments.  
  
from feature_extractors.color_moments import get_color_vector  
  
color_vector = get_color_vector(grid)  
  
# Load color tensor database into memory.  
  
from utils.utilities import tensor_loader  
  
color_tensor_dict = tensor_loader('color_tensors.pt', os.path.abspath(''))
```

```
from utils.utilities import top_k_ranker

# Manhattan distance (City-block distance): This distance can be imagined as the length needed
# to move between two points in a grid where you can only move up, down, left or right. This
# measure is sensitive to differences along each dimension of the feature vectors, capturing
# the total absolute difference between corresponding moments, which makes it great for
# assessing differences in color distribution in terms of absolute deviations. Since it
# does not square the differences like Euclidean distance, it is less susceptible outliers
# when many dimensions differ.

# We use Earth Mover's Distance (Wasserstein distance) as color moments can be thought
# of as probability distributions over color space. EMD is capable of comparing distributions
# with different shapes and scales, color moments consider the spatial arrangement of colors,
# not just the frequency of colors. EMD takes this spatial information into account by
# considering how color probability mass is transported from one distribution to another.

from scipy.spatial.distance import cityblock

print("Similar images by Color Moments using distance: Manhattan (cityblock)")

top_k_by_color = top_k_ranker(K, color_vector, color_tensor_dict, cityblock)

for i in range(len(top_k_by_color)):

    id = top_k_by_color[i][1]
    distance = top_k_by_color[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by Color Moments using distance: Manhattan (cityblock)

1. Image ID: 8676 Distance: 0.0



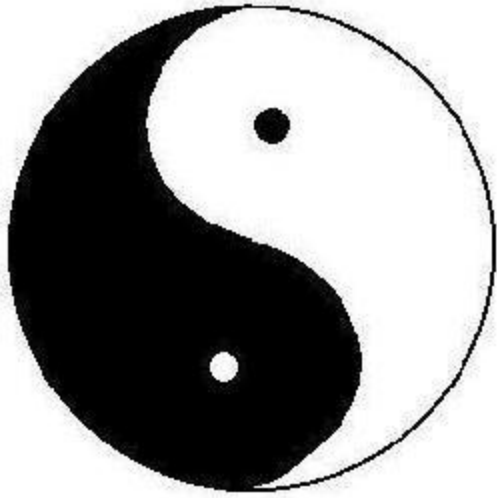
2. Image ID: 8643

Distance: 22167.629



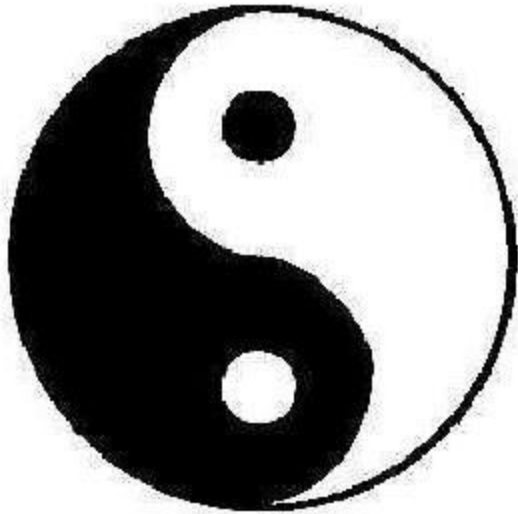
3. Image ID: 8620

Distance: 24031.125



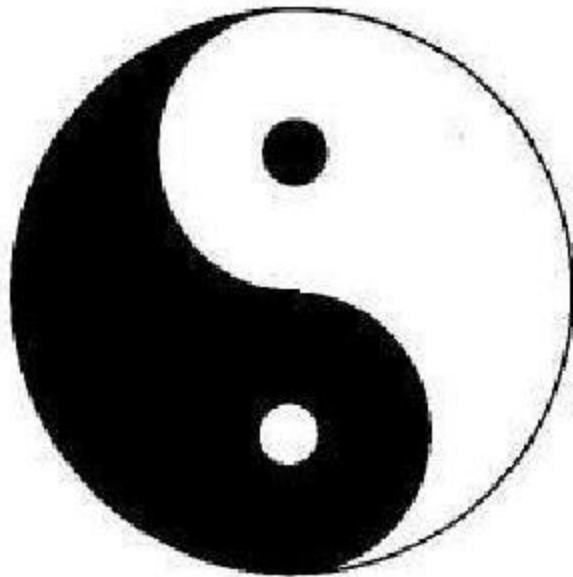
4. Image ID: 8671

Distance: 28903.998



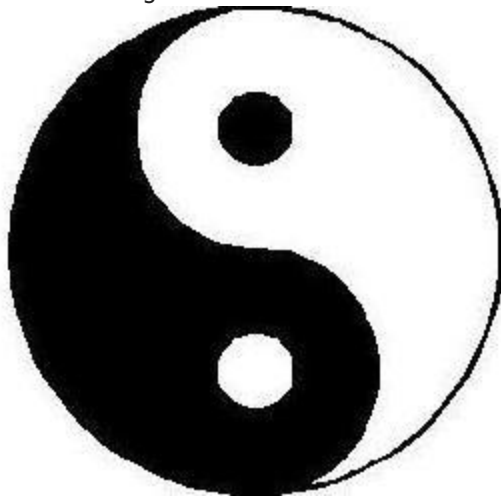
5. Image ID: 8658

Distance: 29158.98



6. Image ID: 8621

Distance: 30705.273



7. Image ID: 8623

Distance: 31181.123



8. Image ID: 5434

Distance: 31207.453



9. Image ID: 8639

Distance: 31974.592



10. Image ID: 6939

Distance: 32399.223



```
In [ ]: # Convert to grayscale and partition to grid.
```

```
from torchvision.transforms import Grayscale
```



```
gs_image = Grayscale()(image300x100)

gs_grid = partition_to_grid(gs_image, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

```
In [ ]: # Histograms of oriented gradients (HOG).

# Find K most similar images by HOG.

from feature_extractors.HOG import get_hog_vector

hog_vector = get_hog_vector(gs_grid)

# Load hog tensor database into memory.

hog_tensor_dict = tensor_loader('hog_tensors.pt', os.path.abspath(""))

# Correlation similarity: This measures the linear relationship between HOG feature vectors,
# and performs well with features that have a strong linear correlation or dependency in
# their gradient orientation distributions, taking into account directionality.

from scipy.spatial.distance import correlation

print("Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation")

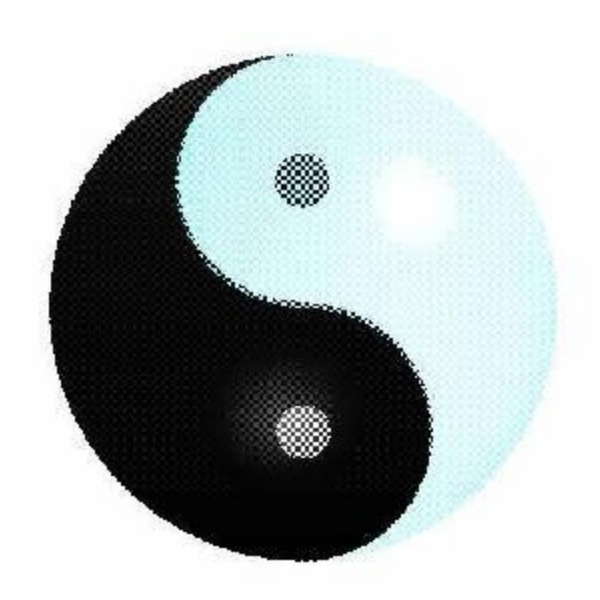
top_k_by_hog = top_k_ranker(K, hog_vector, hog_tensor_dict, correlation)

for i in range(len(top_k_by_hog)):

    id = top_k_by_hog[i][1]
    distance = top_k_by_hog[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation
1.      Image ID: 8676      Distance: 0.0
```



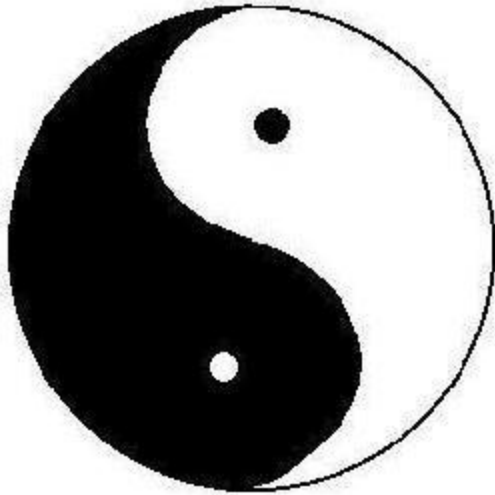
2. Image ID: 8643

Distance: 0.10200639019678914



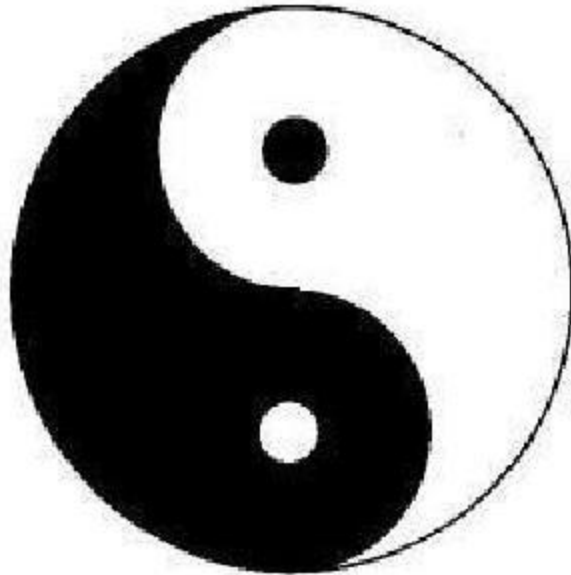
3. Image ID: 8620

Distance: 0.1027280478371535



4. Image ID: 8658

Distance: 0.13705439514590978



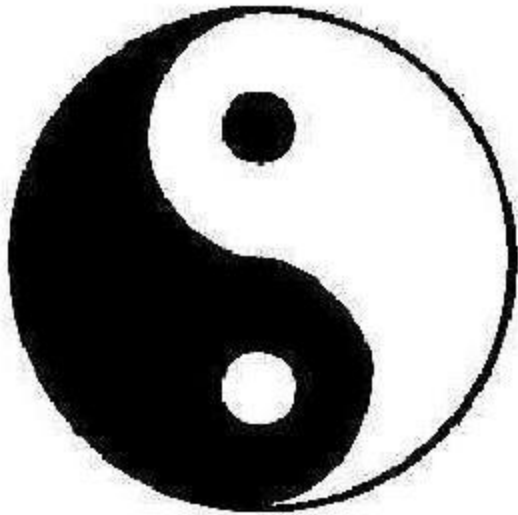
5. Image ID: 8675

Distance: 0.14870996138508585



6. Image ID: 8671

Distance: 0.15401885443272267



7. Image ID: 8667

Distance: 0.15573586599506828



8. Image ID: 8623

Distance: 0.16617783389556773



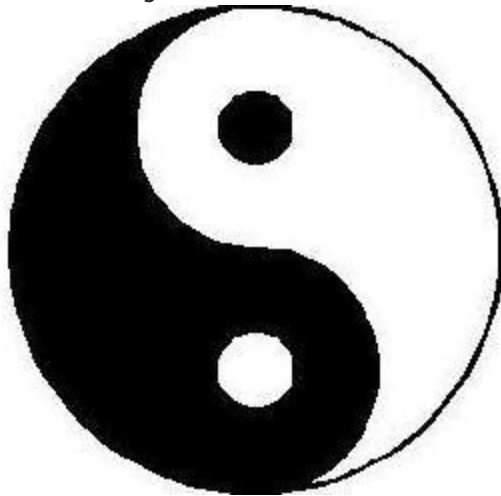
9. Image ID: 8644

Distance: 0.178843138963097



10. Image ID: 8621

Distance: 0.17893489881148705



In []: *# Resize original image to 224x224 and convert to float tensor.*

```
from torchvision.transforms import transforms
```

```
image224x224 = image.resize((224, 224))
```

```
img_tensor = transforms.Compose([transforms.ToTensor()])(image224x224)
```

```
In [ ]: # ResNet pre-processing.

# Find K most similar images by each of the 3 layers chosen from ResNet-50.

from feature_extractors.resnet_50 import get_resnet50_feature_vectors

avgpool_vector, layer3_vector, fc1000_vector = get_resnet50_feature_vectors(img_tensor)

# Load each tensor database into memory

avgpool_tensor_dict = tensor_loader('avgpool_tensors.pt', os.path.abspath(""))
layer3_tensor_dict = tensor_loader('layer3_tensors.pt', os.path.abspath(""))
fc1000_tensor_dict = tensor_loader('fc1000_tensors.pt', os.path.abspath(""))
```

```
In [ ]: # ResNet-AvgPool-1024.

# Find K most similar images by ResNet-AvgPool-1024.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

from scipy.spatial.distance import cosine

print("Similar images by ResNet-AvgPool-1024 using distance: Cosine")

top_k_by_avgpool = top_k_ranker(K, avgpool_vector, avgpool_tensor_dict, cosine)

for i in range(len(top_k_by_avgpool)):

    id = top_k_by_avgpool[i][1]
    distance = top_k_by_avgpool[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

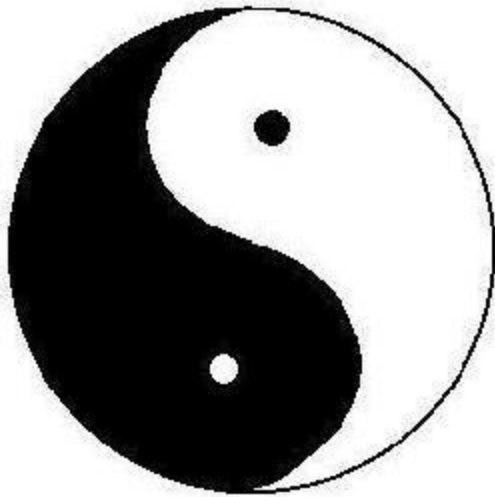
Similar images by ResNet-AvgPool-1024 using distance: Cosine

1. Image ID: 8676 Distance: 0



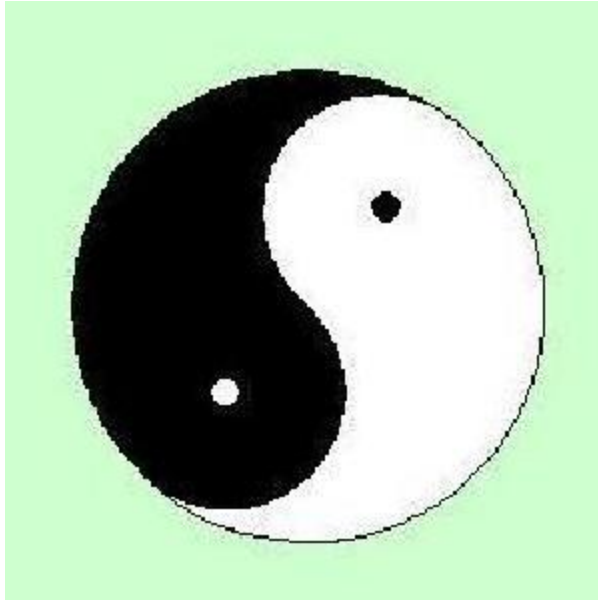
2. Image ID: 8620

Distance: 0.32969796657562256



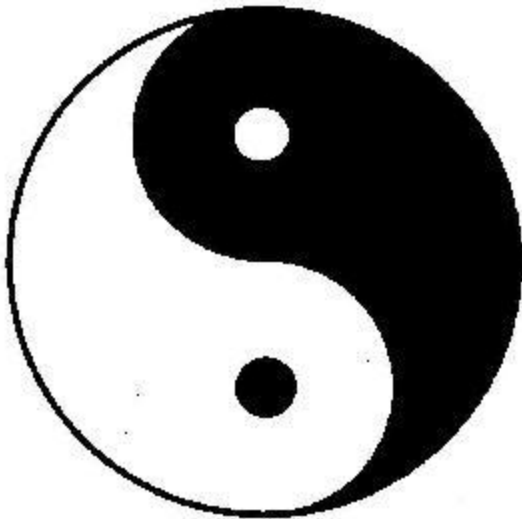
3. Image ID: 8619

Distance: 0.34020692110061646



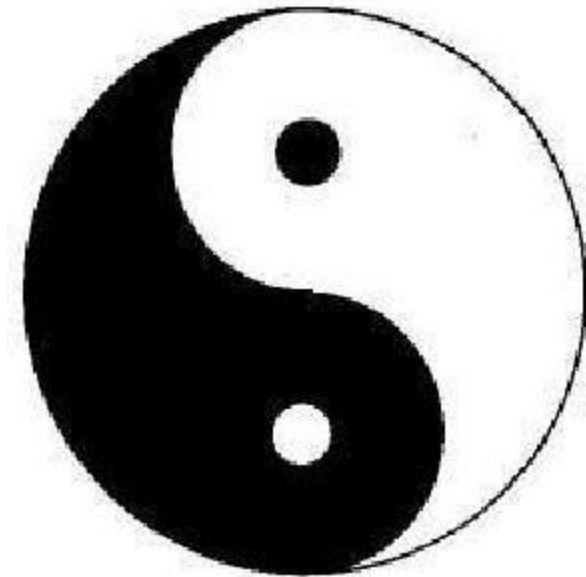
4. Image ID: 8656

Distance: 0.3500450849533081



5. Image ID: 8658

Distance: 0.3510517477989197



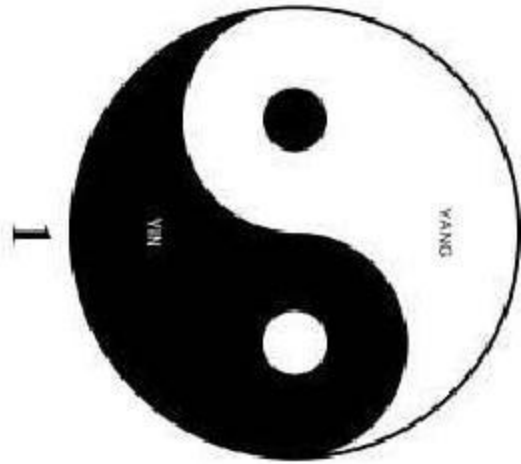
6. Image ID: 8670

Distance: 0.3612073063850403



7. Image ID: 8666

Distance: 0.36177295446395874



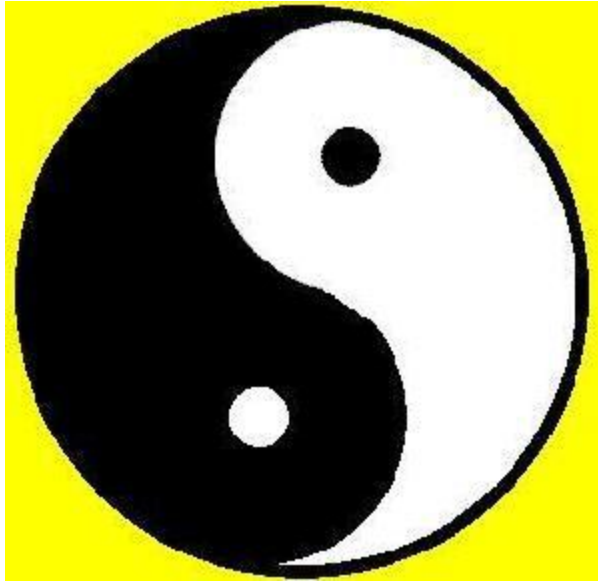
8. Image ID: 8632

Distance: 0.3633463382720947



9. Image ID: 8663

Distance: 0.3829125165939331



10. Image ID: 8634

Distance: 0.3843638300895691



```
In [ ]: # ResNet-Layer3-1024.  
  
# Find K most similar images by ResNet-Layer3-1024.  
  
# Cosine similarity: We use Cosine since the output of the ResNet layer is
```

```

# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

print("Similar images by ResNet-Layer3-1024 using distance: Cosine")

top_k_by_layer3 = top_k_ranker(K, layer3_vector, layer3_tensor_dict, cosine)

for i in range(len(top_k_by_layer3)):

    id = top_k_by_layer3[i][1]
    distance = top_k_by_layer3[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])

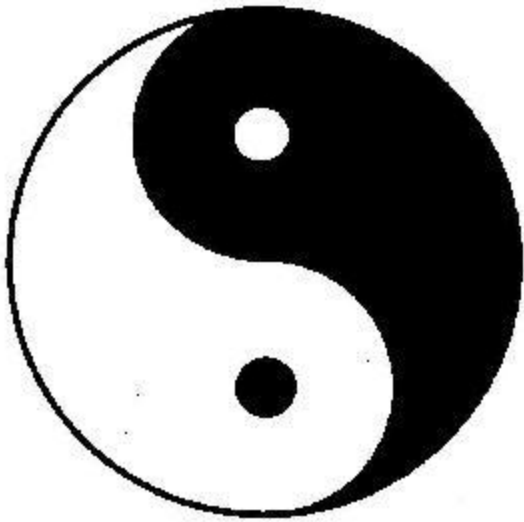
```

Similar images by ResNet-Layer3-1024 using distance: Cosine

1. Image ID: 8676 Distance: 0

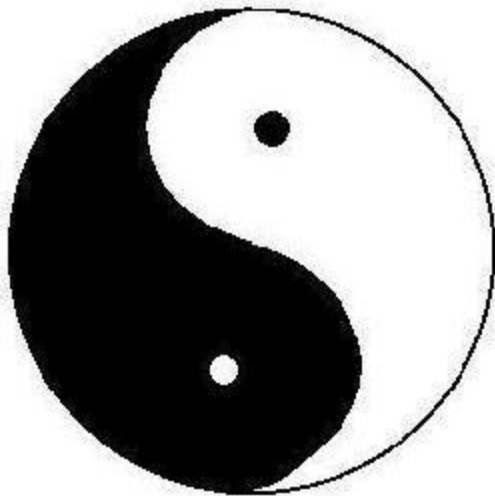


2. Image ID: 8656 Distance: 0.03968888521194458



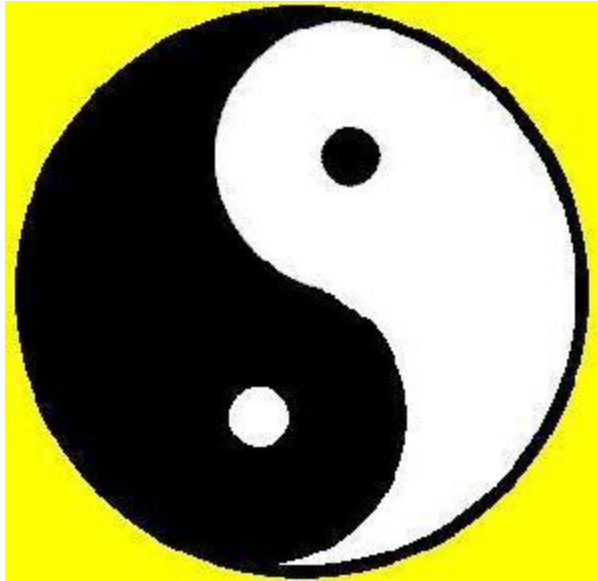
3. Image ID: 8620

Distance: 0.04346984624862671



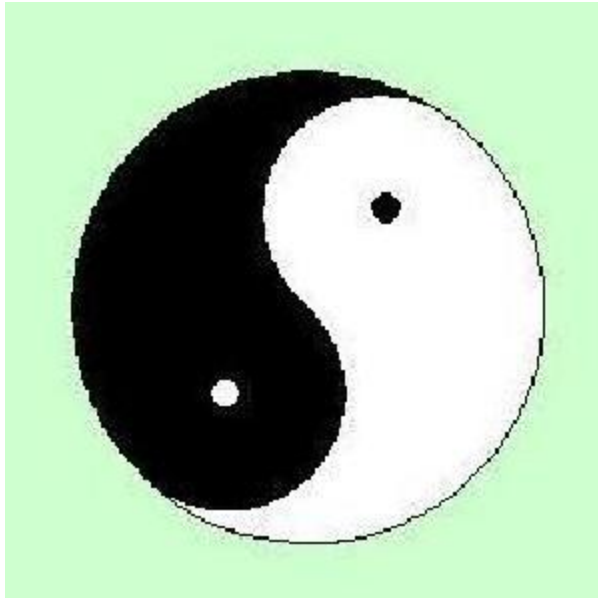
4. Image ID: 8663

Distance: 0.04572659730911255



5. Image ID: 8619

Distance: 0.046192049980163574



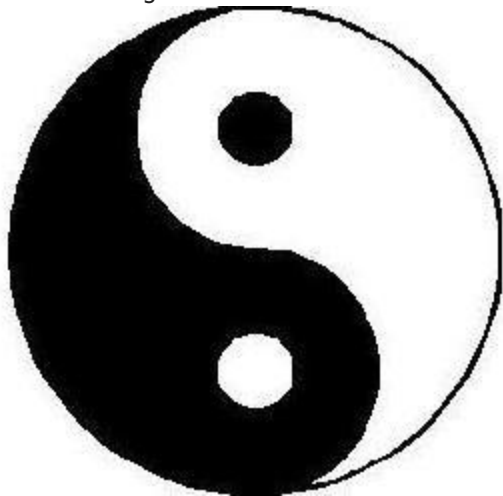
6. Image ID: 8632

Distance: 0.04826962947845459



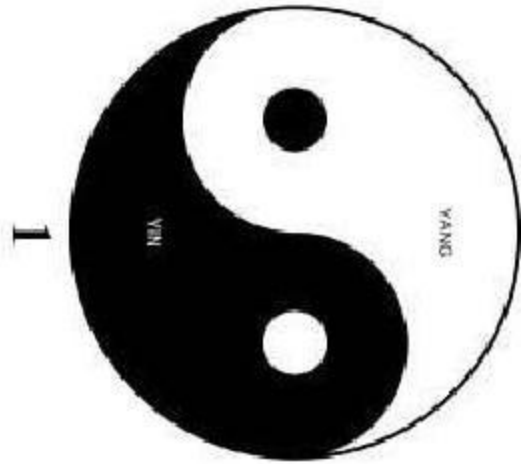
7. Image ID: 8621

Distance: 0.04918557405471802



8. Image ID: 8666

Distance: 0.049281179904937744



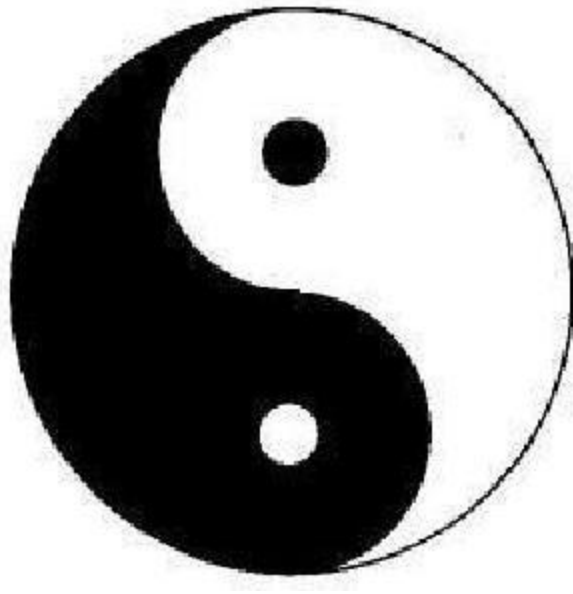
9. Image ID: 8670

Distance: 0.05097758769989014



10. Image ID: 8658

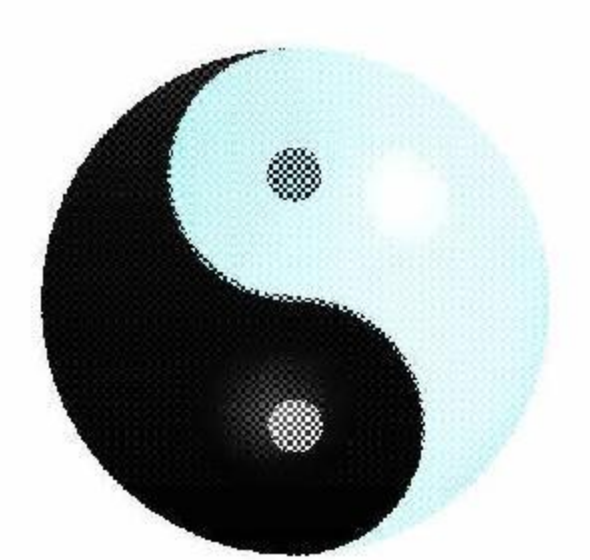
Distance: 0.05378711223602295



```
In [ ]: # ResNet-FC-1000.  
  
# Find K most similar images by ResNet-FC-1000.  
  
# Cosine similarity: We use Cosine since the output of the ResNet layer is  
# normalized, reducing the importance of magnitude. It is good at discerning  
# semantic or structural similarity rather than their absolute feature values.  
# It also works well with sparse feature spaces such as those found in CNNs.  
  
print("Similar images by ResNet-FC-1000 using distance: Cosine")  
  
top_k_by_fc1000 = top_k_ranker(K, fc1000_vector, fc1000_tensor_dict, cosine)  
  
for i in range(len(top_k_by_fc1000)):  
    id = top_k_by_fc1000[i][1]  
    distance = top_k_by_fc1000[i][0]  
  
    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)  
    display(dataset[id][0])
```

Similar images by ResNet-FC-1000 using distance: Cosine

1. Image ID: 8676 Distance: 0



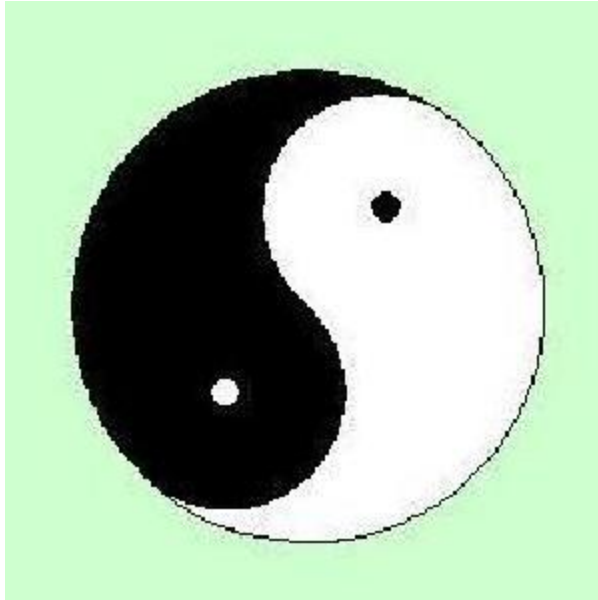
2. Image ID: 8632

Distance: 0.3484852910041809



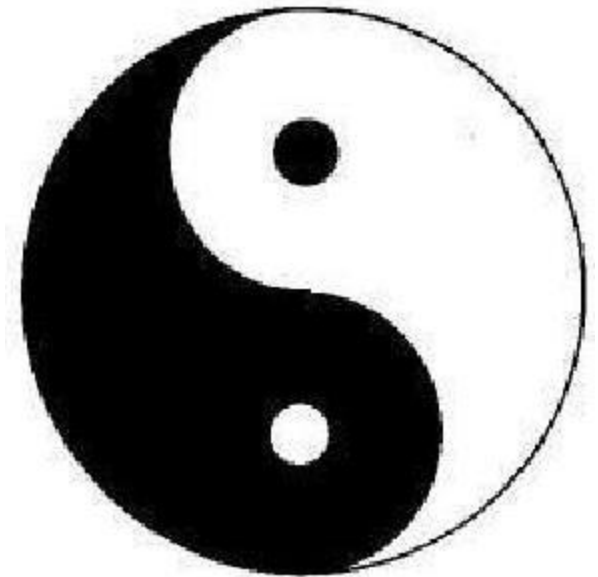
3. Image ID: 8619

Distance: 0.39786356687545776



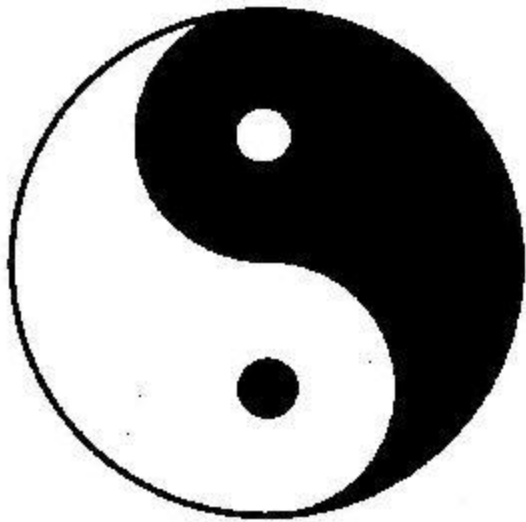
4. Image ID: 8658

Distance: 0.4032387137413025



5. Image ID: 8656

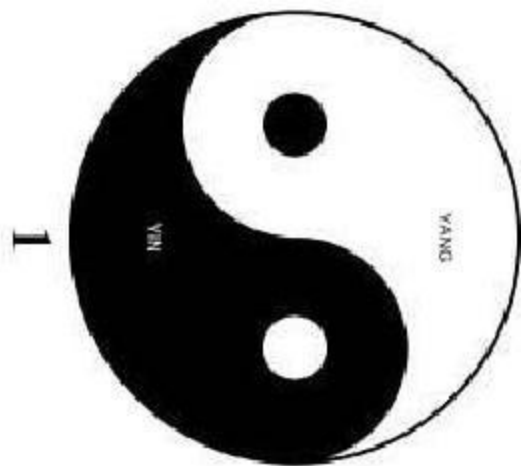
Distance: 0.4055728316307068



6.

Image ID: 8666

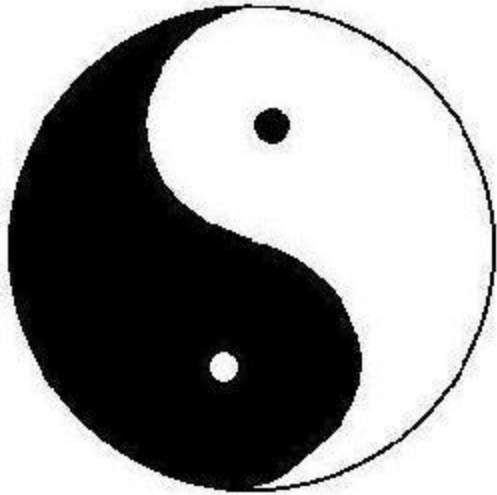
Distance: 0.41052669286727905



7.

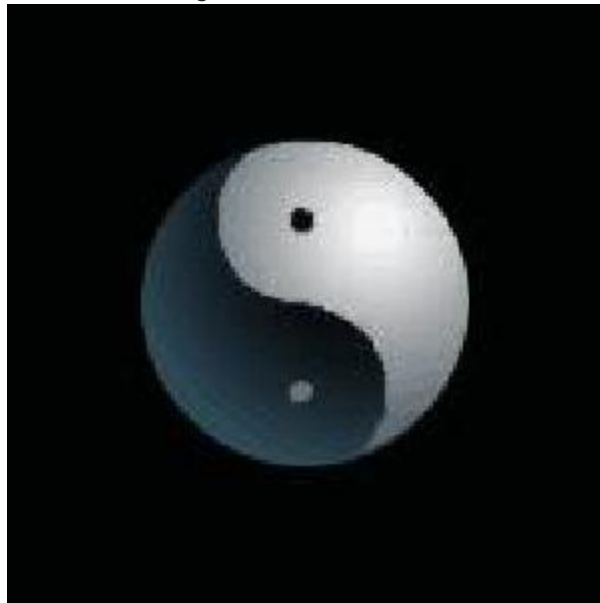
Image ID: 8620

Distance: 0.41611653566360474



8. Image ID: 8668

Distance: 0.4216253161430359



9. Image ID: 8638

Distance: 0.4240154027938843



10. Image ID: 7523

Distance: 0.42618387937545776

